

Section 14.2: Communication & Collaboration Platforms

Part 2 of 8 in the System Design Interview Series

Executive Overview

Real-time communication systems face unique challenges: connection management, message ordering, presence tracking, and offline synchronisation. These systems must maintain consistency while scaling to millions of concurrent users. The key differentiator in interviews is demonstrating understanding of *connection-level* architecture — not just the data model.

1. Design a Chat / Messaging System (WhatsApp / Messenger / Discord)

Requirements Analysis

Dimension	Requirement
Functional	Send/receive messages, group chats, media sharing, read receipts
Scale	1B users, 100M DAU, peak 50M concurrent connections
Latency	< 100ms message delivery
Reliability	Guaranteed delivery; offline message buffering

Connection Architecture

The most important design decision is how to manage 50M persistent connections at peak.

Client → TLS/WebSocket → Load Balancer (Layer 4) → Gateway Node
↓

Connection Registry (Redis)
[user_id → gateway_node_id]

Gateway node design: - Each node handles 10K–100K WebSocket connections - Stateful: knows which users are connected to it - Communicates with other gateways via internal message bus (Kafka/NATS) - Connection metadata stored in Redis:

```
session:{user_id} → {node_id, socket_id, last_seen}
```

Session affinity: Load balancer uses consistent hashing on `user_id` so reconnecting users land on the same gateway (or Redis is used to redirect them).

Message Fan-Out

Fan-out-on-Write (group messages):

Sender → API Server → Fan-out Service

↓

For each group member:

1. Write to member's message queue (Kafka partition by user_id)
2. Gateway delivers via WebSocket if online
3. Push notification service if offline

Fan-out-on-Read (alternative for large groups):

Sender → Writes message centrally (Cassandra, keyed by group_id + timestamp)
Reader → Fetches group messages on open + last-read cursor

Approach	Write Cost	Read Cost	Best For
Fan-out-on-write	O(members)	O(1) — pre-delivered	Small to medium groups (≤ 1000)
Fan-out-on-read	O(1)	O(new messages since cursor)	Large groups, public channels
Hybrid	Variable	Variable	General-purpose messenger

Connection Recovery

Clients maintain a `last_received_message_id`. On reconnect: 1. Client sends `RECONNECT {user_id, last_msg_id}` 2. Server fetches all messages in conversation with sequence $> \text{last_msg_id}$ 3. Deliver in order; client deduplicates by message ID

```
Cassandra schema:
PRIMARY KEY ((conversation_id), sequence_number)
CLUSTERING ORDER BY sequence_number DESC
```

This enables efficient range queries: `SELECT * WHERE conversation_id=X AND sequence_number > Y`.

Presence Tracking

```
User comes online:
→ Redis SET "presence:{user_id}" = "online" EX 90 (90s TTL)
→ Heartbeat every 30s resets TTL

User goes offline (or heartbeat missed):
→ TTL expires → key deleted
→ Presence service detects deletion → broadcasts "offline" to relevant contacts
```

Scaling presence: For 100M DAU, storing `{user_id → status}` in Redis cluster is ~3GB (trivial). Broadcasting presence updates to all followers is the expensive part — use a pub-sub fan-out capped by friend count.

Message State Machine

```
SENT (client) → DELIVERED (server received) → DELIVERED (recipient device) → READ
(recipient opened)
```

Each state transition emits an event back to the sender. Read receipts are batched (send once per 5 seconds) to avoid write amplification.

End-to-End Encryption (Signal Protocol Overview)

1. Each user pre-uploads a bundle of one-time pre-keys to the server
2. Sender fetches recipient's pre-key bundle, performs X3DH key exchange → derives shared secret
3. Double ratchet algorithm generates a new encryption key for every message
4. Server stores only ciphertext — cannot decrypt messages

Interview implication: E2EE means server-side search is impossible. If full-text search is required, it must happen client-side (index stored locally, encrypted).

Interview Problem 1 — WhatsApp Group Message Ordering

“In a WhatsApp-style system, two users send messages to the same group at the same time from different regions. How do you ensure all members see messages in the same order?”

Solution framework:

1. **Total order via per-conversation sequencer:** Each conversation has a logical sequence counter stored in a single authoritative node (or distributed counter via Redis `INCR`). Every message must acquire a sequence number before delivery.
 2. **Implementation:** `INCR conversation:{conv_id}:seq` returns monotonically increasing integers. Assign to message before writing to Cassandra.
 3. **Client ordering:** Clients sort messages by `sequence_number`, not client timestamp. Client timestamps are only for display.
 4. **Cross-region challenge:** If the sequencer is in US-East and a user in Asia sends a message, they pay ~150ms to get a sequence number. Mitigation: Accept slight reordering within a few seconds; clients reconcile on next sync.
 5. **Trade-off stated:** Perfect global ordering requires global coordination (slow). Casual ordering (causally consistent, eventually ordered) is what WhatsApp actually uses — good enough for human conversation.
-

Interview Problem 2 — Scaling to 10M Concurrent WebSocket Connections

“Your messaging platform needs to support 10M concurrent WebSocket connections. How do you architect this, and how do you handle gateway node failures?”

Solution framework:

1. **Horizontal gateway fleet:** 1,000 gateway nodes × 10K connections each = 10M. Each node is a stateless process (state is in Redis).
 2. **Failure detection:** Each node publishes a heartbeat to Redis every 5s. If heartbeat stops for 15s, node is considered dead. A supervisor process marks the node down.
 3. **Client reconnect:** Clients detect WebSocket disconnect within 30s (keep-alive). On reconnect, they hit the load balancer which routes to a healthy node. They re-register session in Redis.
 4. **In-flight messages:** Messages queued for offline delivery in Kafka (topic `pending:{user_id}`). Once user reconnects and session is restored, messages drain from Kafka.
 5. **Zero message loss guarantee:** Producers use Kafka `acks=all` + idempotent producer. Messages are retained for 7 days, allowing reconnect recovery.
-

2. Design Real-Time Collaboration (Google Docs Style)

Requirements Analysis

Dimension	Requirement
Functional	Concurrent editing by multiple users, conflict-free merge
Scale	Millions of documents; thousands of concurrent editors per document
Latency	< 50ms update propagation

Approach Comparison: OT vs CRDT

Approach	Correctness	Availability	Complexity	Used By
Operational Transform (OT)	Perfect — central server transforms all ops	Poor (central server = SPOF)	Medium	Google Docs (historically)
CRDT	Perfect — convergence guaranteed without coordination	Excellent — works offline	High	Figma, Linear, many modern tools

Operational Transforms (OT) — How It Works

Two users concurrently edit the same document:

Initial: "Hello World"

User A: Insert "!" at position 11 → "Hello World!"

User B: Delete character at position 6 → "HelloWorld"

Without transform: Applying B after A deletes wrong character (index shifted)

With OT server:

- Server receives A's op first → applies it → document = "Hello World!"

- Server transforms B's op: position 6 in new doc is still "W" → applies deletion

- Result: "HelloWorld!" — both intentions preserved

CRDTs — How They Work

Each character is assigned a unique, immutable ID. Insertions/deletions reference these IDs, not positions.

```
User A inserts "!" after char-ID:42 (after "d")
User B deletes char-ID:37 ("o")
→ Both operations are independent and commutative
→ Apply in any order → same result
```

Deletions use **tombstones**: the character is marked deleted but remains in the structure to preserve relative ordering. Garbage collection removes old tombstones in background.

Interview Problem 3 — Offline Editing Sync

“A user edits a Google Doc for 10 minutes while offline, then reconnects. How do you merge their changes with edits made by others during that time?”

Solution framework:

1. **Client-side operation log**: While offline, client accumulates all operations in a local log with a Lamport timestamp.
2. **On reconnect**: Client sends the full operation log to the collaboration server.
3. **Server-side transformation** (OT approach): Server replays client's operations, transforming each one against the operations that were applied by other users during the offline period. This produces a final merged document state.
4. **CRDT approach**: No server transformation needed. The client's operations are simply broadcast to all active clients. Because CRDTs are commutative, everyone converges to the same state.
5. **Conflict presentation**: For semantic conflicts (two users changed the same sentence), present a visual diff to the user (“User B also changed this paragraph while you were offline”).

3. Design a Video Conferencing System (Zoom / Google Meet)

Architecture Pattern Comparison

Pattern	Bandwidth Model	Server Cost	Scale Limit	When to Use
Mesh (P2P)		None	~6 participants	

Pattern	Bandwidth Model	Server Cost	Scale Limit	When to Use
	$O(N^2)$ — every participant uploads $N-1$ streams			Small calls, no server budget
SFU (Selective Forwarding Unit)	$O(N)$ uploads, $O(N)$ downloads	Medium	Hundreds of participants	Standard video conferencing
MCU (Multipoint Control Unit)	$O(N)$ uploads, $O(1)$ download	High (encode/decode)	Tens of participants	Bandwidth-constrained clients, legacy

SFU is the standard production choice. Each participant sends one stream to the SFU; the SFU forwards streams selectively (active speaker, layout preference) without re-encoding.

WebRTC NAT Traversal

The three-step process for establishing P2P or SFU connections:

1. **STUN**: Client contacts STUN server to discover its public IP:port mapping (reflexive address)
2. **ICE candidate gathering**: Client collects all possible connection candidates (host, STUN-derived, TURN-relay)
3. **TURN fallback**: When direct P2P fails (symmetric NAT), relay all traffic through TURN server (~10% of calls)

Adaptive Bitrate for Video Calls

Unlike streaming (pre-recorded), video calls must adapt in real time: - **REMB / TWCC**: Receiver sends bandwidth estimation feedback every 200ms - **Simulcast**: Sender simultaneously transmits 3 resolution layers (e.g., 1080p, 720p, 360p). SFU selects which layer to forward per receiver based on their available bandwidth - **Frame dropping vs resolution reduction**: Resolution reduction is preferred (less disruptive perceptually) until bandwidth is critically low, then frame rate drops

Section Summary: Communication Platform Patterns

Challenge	Pattern	Notes
50M+ WebSocket connections	Fleet of gateway nodes + Redis session registry	10K–100K per node

Challenge	Pattern	Notes
Message ordering	Per-conversation sequence number (Redis INCR)	Accept eventual order for cross-region
Concurrent document editing	CRDT for availability; OT if central authority is acceptable	CRDT preferred for modern systems
Video call architecture	SFU for most cases	MCU only for bandwidth-constrained scenarios
Offline sync	Client operation log + replay on reconnect	OT transforms vs CRDT merge